DSA-II CASE STUDY ASSIGNMENT

Pre-End Semester Assessment

Course: Data Structures and Algorithms-II (DSA-II)
Purpose: Apply the algorithms studied in class to a simple real-world problem.

1. Instructions for Students

- Choose ONE topic from the list below. Topic selection should be approved by the faculty member.
- Every student must complete and submit the case study individually.
- Keep the work at an easy-to-medium level. Use small examples and explain the algorithm in your own words.
- Include a C++ implementation for algorithm-based topics. The code should be understandable and should run for at least a few test cases.
- Do not attempt to build a complete commercial application. A small demonstration or console-based program is sufficient.
- Use reliable references and acknowledge any external code, diagrams, or AI assistance according to course instructions.
- Submit the completed report by 4 November.

2. Approved Case Study Topics

No.	Topic and Suggested Scope	Relevant Algorithms / Concepts
1	University Examination Timetable Scheduling Represent courses as vertices and conflicts as edges. Assign examination slots so that conflicting courses are not scheduled at the same time.	Graph Coloring, Backtracking
2	Optimizing Cargo Loading for Delivery Vehicles Select items for a vehicle with a limited capacity to maximize total value. Use a small example to compare two approaches.	Fractional Knapsack, 0/1 Knapsack, Dynamic Programming
3	Document Similarity and Plagiarism Detection Compare two short text samples and demonstrate how an algorithm can measure common content or editing differences. Explain that similarity alone does not prove plagiarism.	Longest Common Subsequence (LCS), Edit Distance, Dynamic Programming
4	Shortest Path Planning for Emergency Services Represent locations as vertices and roads as weighted edges. Find the shortest route between a starting point and a destination.	Dijkstra's Algorithm; optionally compare with Bellman–Ford
5	Cost-Effective Network Infrastructure Design Connect buildings or computer labs with minimum total cable cost while ensuring every location is connected.	Prim's Algorithm, Kruskal's Algorithm, Union-Find

6	Project Task Scheduling Using Dependencies Represent tasks and their dependencies as a directed graph. Find a valid task order and demonstrate how a cycle can prevent scheduling.	Topological Sorting using BFS or DFS, Cycle Detection
7	Efficient Keyword Search in Large Text Documents Search for a word or short pattern in a text. Compare the number of character comparisons for a small example.	Naïve String Matching, KMP or Rabin–Karp
8	Delivery Route Optimization for Multiple Destinations For a small number of destinations, find a low-cost tour that visits each destination and returns to the start. Discuss why the problem becomes harder as destinations increase.	Travelling Salesperson Problem (TSP), basic exhaustive search or Branch and Bound
9	Data Compression for Efficient File Storage Build a Huffman tree for a short sample message and compare the encoded length with fixed-length coding.	Huffman Coding, Greedy Strategy
10	Finding Subsets That Match a Target Sum Given a small set of numbers and a target, find whether a subset adds up to the target. Show the search or DP process.	Subset Sum, Backtracking or Dynamic Programming

3. Report Format

Suggested length: 6–10 pages, excluding the cover page and references. Keep explanations clear and concise.

1. **Cover Page:** Case study title, student name(s), UID(s), course name/code, faculty name, department, and submission date.
2. **Abstract (100–150 words):** Briefly state the problem, algorithm used, approach, and main finding.
3. **Introduction and Problem Statement:** Explain the real-world situation and the specific problem being solved.
4. **Objectives:** List 3–4 clear objectives.
5. **Problem Modelling:** Identify the input, output, constraints, and assumptions. Show how the problem is represented using arrays, graphs, strings, weights, or other suitable structures.
6. **Algorithm Explanation:** Explain the selected algorithm in simple steps. Include pseudocode or a small flowchart if useful.
7. **Worked Example:** Use a small sample and show the important steps, such as a graph trace, DP table, matching steps, or subset selection.
8. **C++ Implementation and Testing:** Include readable code, sample input/output, and at least 2–3 test cases. A console-based program is enough.
9. **Complexity Analysis:** State the time and space complexity using Big-O notation and explain what it means for the chosen example.
10. **Results and Discussion:** Explain what the output shows. If comparing algorithms, use a small table and discuss the differences.

11. **Conclusion and Limitations:** Summarize what was learned and mention one limitation or possible improvement.
12. **References and Acknowledgement:** List books, websites, papers, or other material used. Acknowledge any reused code or AI assistance as required.

4. Minimum Technical Requirements

- At least one clear worked example related to the chosen topic.
- A C++ program or small implementation/demo, with sample output.
- At least 2–3 test cases, including a normal case and a simple edge case where relevant.
- Time and space complexity in Big-O notation, explained in simple terms.
- A comparison with another approach where suitable; a full comparison is not compulsory for every topic.
- Students must be able to explain the algorithm and code during the viva.

5. Suggested Evaluation Scheme (30 Marks)

Evaluation Criterion	Marks
Problem statement, relevance, and objectives	3
Problem modelling and algorithm selection	4
Algorithm explanation and worked example	5
Implementation, testing, and sample output	9
Time/space complexity and discussion	5
Conclusion, report quality, and references	4
Total	30